WalangBrownout Local Setup Guide

The project is finished and working, so this guide only gets it running on your computer. Use PowerShell, paste one box at a time, press Enter, and wait for it to finish before the next one.

1. Before you start

Install Docker Desktop and Node.js (LTS) if you do not have them. Then open Docker Desktop and wait until it says it is running.

Check Docker:

```
docker --version
```

Check Docker Compose:

```
docker compose version
```

Check Node:

```
node -v
```

Check npm:

```
npm -v
```

Each one should print a version number. If one says "not recognized", that program is not installed yet.

2. Unzip and check the folder layout

Unzip the project so it looks like this:

```
WalangBrownout/
  case-study-frontend/
  docker-laravel/
```

Go into the docker-laravel folder (change the path to wherever you unzipped it):

```
cd C:\Users\yourname\Downloads\WalangBrownout\docker-laravel
```

List the backend files:

```
dir src
```

You must see `artisan` and `composer.json` in that list. If you only see one other folder (like `case-study-backend`), the files are nested one level too deep. Move everything from inside that folder up into `src`.

3. Start the backend containers

Stay in the docker-laravel folder. If you closed PowerShell, repeat the `cd` line from step 2. Then:

```
docker compose up --build -d
```

The first build takes a few minutes. It is done when `laravel-app`, `laravel-db` and `laravel-pma` all show Started or Running.

4. Install and set up the backend

Still in the docker-laravel folder. Install the PHP packages (it can look stuck for a minute or two, let it finish):

```
docker compose exec app composer install
```

Only if `APP_KEY=` in `.env` is empty, generate a key:

```
docker compose exec app php artisan key:generate
```

Create the tables and the demo data:

```
docker compose exec app php artisan migrate:fresh --seed
```

If this says it cannot connect to the database, MySQL is still starting. Wait 15 seconds and paste the same line again.

5. Check the backend

Open these in your browser:

- http://localhost:8000 shows the Laravel welcome page.
- http://localhost:8083 opens phpMyAdmin (user `root`, password `root`). Open the `casestudy` database and confirm the tables have data.

Then paste these three lines into PowerShell together:

```
$r = Invoke-RestMethod -Method Post -Uri http://localhost:8000/api/login -
ContentType "application/json" -Body
'{"email":"mark.nathan@walangbrownout.test","password":"warehouse2026"}'
$h = @{ Authorization = "Bearer $($r.token)"; Accept = "application/json" }
Invoke-RestMethod -Uri http://localhost:8000/api/workspace -Headers $h
```

If it prints products, lots, alerts, activity and tasks, the backend works.

6. Set up the frontend

Go into the frontend folder (change the path to match yours):

```
cd C:\Users\yourname\Downloads\WalangBrownout\case-study-frontend
```

Create the `.env` file next to `package.json` (not inside `src`). This one line creates it:

```
Set-Content .env "VITE_API_URL=http://localhost:8000/api"
```

Install the packages (the "high severity vulnerabilities" warning is normal, ignore it):

```
npm install
```

Start the app:

```
npm run dev
```

Open the link it prints (usually http://localhost:5173). Keep this PowerShell window open while you use the app.

7. Log in and final check

Log in with:

```
mark.nathan@walangbrownout.test
warehouse2026
```

If you see the dashboard with products and alerts, the setup is complete. The first load can take a second or two, that is normal. Refreshing the page sends you back to the login screen, that is also normal.